

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №6
по дисциплине
«Программирование»

Выполнил студент гр. ИВТб-1303-06-00

_____/Гортоломей И.К./

Проверил преподаватель кафедры ЭВМ

_____/Баташев П.А./

Киров

2026

Цель работы

Освоить методы векторизованной предобработки данных с использованием библиотеки NumPy. Получить навыки работы с структурированными массивами, булевыми масками, групповой агрегацией и созданием производных признаков для задач промышленного мониторинга.

Постановка задачи

Необходимо выполнить предобработку данных с датчиков очистных сооружений (вариант 20). Исходный файл `data.csv` содержит следующие поля:

- `ts` (`int32`) — время замера;
- `filter_id` (`int16`) — идентификатор фильтра;
- `turb` (`float32`) — турбидность, NTU;
- `ph` (`float32`) — кислотность воды;
- `cl` (`float32`) — остаточный хлор, мг/л;
- `flow` (`float32`) — расход воды, м³/ч.

Требуется реализовать 10 этапов обработки:

1. Загрузка данных в структурированный массив `numpy.dtype`, оценка объёма памяти, подсчёт NaN/Inf.
2. Векторизованная фильтрация аномалий: `turb < 0 → 0`, `cl < 0 → 0`, `ph → clip[6.5, 8.5]`.
3. Группировка по `filter_id`, расчёт статистик и Z-score нормализация `turb` внутри групп.
4. Скользящее среднее для `turb` (окно $k = 35$) через кумулятивную сумму, вычисление `np.diff(flow)`.

5. Создание производных признаков: индекс чистоты и эффективность хлорирования.
6. Условная агрегация: статистики по `turb` для записей с расходом выше среднего.
7. Лаговые признаки (`lag=1`) для `turb`, анализ распределения знаков разницы.
8. Робастная замена выбросов по методу IQR внутри каждой группы фильтров.
9. Проверка согласованности данных по предметным правилам, замена некорректных значений.
10. Частотный анализ категориального поля `filter_id`, объединение редких категорий.

Ограничения: запрещено использование циклов `for` по строкам исходного массива. Все операции должны выполняться векторизованно через булевы маски, `np.where`, `np.clip`, `np.unique`.

Описание реализации

Обработка выполнена в среде Google Colab с использованием библиотеки NumPy. Код организован в виде последовательных ячеек, каждая из которых соответствует одному этапу лабораторной работы.

Этап 1: Загрузка и подготовка типов

Для чтения файла использована функция `np.genfromtxt` с явно заданным структурированным типом данных:

```
1 dtype = np.dtype([
2     ('ts', np.int32), ('filter_id', np.int16),
3     ('turb', np.float32), ('ph', np.float32),
4     ('cl', np.float32), ('flow', np.float32)
5 ])
6 data = np.genfromtxt('/content/data.csv', delimiter=',',
7     skip_header=1, dtype=dtype, encoding='utf-8')
```

Объём памяти рассчитан через атрибут `nbytes`, подсчёт некорректных значений выполнен через `np.isfinite`.

Этап 2: Векторизованная очистка

Аномалии обнаружены составной булевой маской:

```
1 mask_anomaly = (data['turb'] < 0) | (data['cl'] < 0) | \
2     (data['ph'] < 6.5) | (data['ph'] > 8.5) | \
3     ~np.isfinite(data['turb']) | ~np.isfinite(data['cl'])
```

Очистка выполнена без циклов:

```
1 data['turb'] = np.where(data['turb'] < 0, 0.0, data['turb'])
2 data['cl']   = np.where(data['cl'] < 0, 0.0, data['cl'])
3 data['ph']   = np.clip(data['ph'], 6.5, 8.5)
```

Этап 3: Группировка и нормализация

Группировка реализована через `np.unique` с циклом по уникальным значениям `filter_id` (цикл по группам разрешён). Z-score нормализация:

$$z = \frac{x - \mu_{group}}{\sigma_{group} + \varepsilon}, \quad \varepsilon = 10^{-8} \quad (1)$$

Этап 4: Скользящее окно и разность

Скользящее среднее вычислено за $O(N)$ через кумулятивную сумму:

```
1 cs = np.cumsum(turb_arr)
2 mavg[k-1:] = (cs[k-1:] - np.concatenate(([0.0], cs[:-k]))) / k
```

Изменение расхода: `diff_flow = np.insert(np.diff(data['flow']), 0, 0.0)`.

Этапы 5–10

- Производные признаки: `clean_idx = 1.0 / (turb + 1e-8)`, `cl_eff = cl / (flow + 1e-8)`.
- Условная агрегация: составная маска `(filter_id == fid) & (flow > mean_flow)`.
- Лаговые признаки: `lag1 = np.roll(turb, 1)`, анализ через `np.sign` и `np.bincount`.
- IQR-замена: границы $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$ вычислены внутри групп.
- Проверка логики: предметные правила для питьевой воды, замена через `np.where`.
- Сжатие категорий: редкие `filter_id` (<1% вхождений) объединены в `OTHER` (код 0).

Результаты выполнения

Обработка выполнена на наборе из 2 000 000 записей. Основные метрики представлены в таблице 1.

Анализ временных сдвигов (`lag=1` для турбидности):

- Рост значения: 38.25% записей;
- Падение значения: 38.22% записей;

Таблица 1: Результаты предобработки данных

Показатель	Значение	Комментарий
Объём данных	41.96 МБ	2 млн строк × 6 полей
Доля NaN/Inf	3.00%	Превышает порог 3%, требует внимания
Строк с аномалиями	98.74%	Характерно для «сырых» SCADA-данных
Количество групп	100	Равномерное распределение по фильтрам
Заменено выбросов (IQR)	6.54%	1300 значений на фильтр в среднем
Нарушений логики	36.29%	После первичной очистки
Редких категорий	53 из 100	Объединены в OTHER (код 0)

- Без изменений: 23.53% записей.

Симметричная динамика указывает на стабильный технологический цикл с естественными колебаниями нагрузки.

Условная агрегация по группам (расход выше среднего) показала, что среднее значение турбидности в режиме повышенной нагрузки варьируется в пределах 6.35–7.15 NTU, что соответствует нормативным требованиям для питьевой воды.

Выводы

В ходе выполнения лабораторной работы были освоены методы векторизованной предобработки данных с использованием библиотеки NumPy. Ключевые результаты:

1. Реализована полная обработка массива из 2 млн записей без циклов по строкам, что обеспечило высокую производительность.
2. Применены предметно-ориентированные правила очистки: физические ограничения для турбидности и хлора, санитарные нормы для pH.

3. Групповая нормализация и IQR-замена выбросов позволили учесть технологическую неоднородность оборудования.
4. Временные признаки (скользящее среднее, лаги, разности) выявили устойчивые тренды качества воды и динамику работы насосов.
5. Сжатие редких категорий предотвратило разреженность признакового пространства для последующего машинного обучения.

Особенности предобработки данных водоподготовки:

- Высокий уровень аппаратного шума требует агрессивной, но физически обоснованной фильтрации.
- Жёсткие санитарные нормативы (pH 6.5–8.5) диктуют обязательное клипирование, а не статистическую замену.
- Групповая обработка критически важна: разные фильтры имеют разную пропускную способность и степень износа.
- Векторизованные операции на базе NumPy позволяют обрабатывать миллионы записей за секунды, что необходимо для систем реального времени.

Итоговый массив `data_final.npy` физически непротиворечив, нормализован и готов к решению прикладных задач: прогнозированию качества воды, мониторингу соответствия СанПиН, предиктивному обслуживанию оборудования.

Ссылка на рабочую среду

Код лабораторной работы размещён в среде Google Colab:

https://colab.research.google.com/drive/1VEUo_OGYymcWpvtSMHdohwfX050kf_bB?usp=sharing

Листинги кода

Фрагмент: векторизованная очистка (Этап 2)

```
1 # Поиск аномалий составной маской
2 mask_anomaly = (data['turb'] < 0) | (data['cl'] < 0) | \
3                 (data['ph'] < 6.5) | (data['ph'] > 8.5) | \
4                 ~np.isfinite(data['turb']) | ~np.isfinite(data['cl'])
5
6 # Предварительная замена NaN/Inf медианами
7 for f in ['turb', 'ph', 'cl', 'flow']:
8     col = data[f]
9     bad = ~np.isfinite(col)
10    if np.any(bad):
11        col[bad] = np.nanmedian(col)
12
13 # Очистка согласно заданию (без циклов по строкам)
14 data['turb'] = np.where(data['turb'] < 0, 0.0, data['turb'])
15 data['cl']    = np.where(data['cl'] < 0, 0.0, data['cl'])
16 data['ph']    = np.clip(data['ph'], 6.5, 8.5)
```

Фрагмент: групповая IQR-замена (Этап 8)

```
1 turb_iqr = data_fe['turb'].copy()
2 for fid in unique_filt:
3     m = data_fe['filter_id'] == fid
4     vals = turb_iqr[m]
5     q1, q3 = np.percentile(vals, [25, 75])
6     iqr = q3 - q1
7     lower, upper = q1 - 1.5*iqr, q3 + 1.5*iqr
8     outlier = (vals < lower) | (vals > upper)
9     if np.any(outlier):
10        turb_iqr[m][outlier] = np.median(vals)
11 data_fe['turb'] = turb_iqr
```